

Universidad Siglo 21

Ingeniería en Software

Seminario Final

Módulo 2: Resultados

Trabajo Final de Graduación: Trabajo de Investigación

Tema de investigación:

**Interfaces cerebro-computadora bajo condiciones adversas: del
software a la decodificación**

Alumno: Agustín Tamagusuku

Legajo: SOF02612

Profesor tutor: Luis Miguel Nuñez

Fecha de entrega: 28 de septiembre de 2026

Introducción

Una interfaz cerebro-computadora (BCI, por su sigla en inglés) es un sistema que mide la actividad del sistema nervioso central y la convierte en una salida artificial, por ejemplo un texto en pantalla o el movimiento de un cursor, que reemplaza o complementa una salida natural del cuerpo (Wolpaw et al., 2002). Tiene aplicaciones en investigación con personas sanas y en neurorrehabilitación, pero uno de sus principales usos clínicos, y el que enmarca este trabajo, es la comunicación asistiva para personas con parálisis severa que conservan la capacidad de pensar pero no la de moverse o hablar. Los resultados recientes muestran hasta dónde llegó el campo: Willett et al. (2023) reportaron una neuroprótesis del habla, un dispositivo implantado que reemplaza la función de hablar, que permitió a una participante con esclerosis lateral amiotrófica, una enfermedad neurodegenerativa que paraliza progresivamente los músculos, producir texto a 62 palabras por minuto a partir de su actividad cortical, Metzger et al. (2023) describieron un sistema que decodificó habla y expresiones faciales para controlar un avatar digital, y Card et al. (2024) mostraron que un sistema comparable pudo calibrarse en pocas sesiones con tasas de error de palabra inferiores al 3 %.

Para entender qué se estudia en este trabajo conviene seguir el recorrido de la señal. Nicolas-Alonso y Gomez-Gil (2012) describieron las etapas de toda BCI: la adquisición, en la que electrodos captan la actividad eléctrica del cerebro, en la variante no invasiva mediante electroencefalografía (EEG) sobre el cuero cabelludo, donde cada electrodo es un canal y cada valor de voltaje que se registra, cientos de veces por segundo, es una muestra; el transporte, que lleva esa señal desde el equipo de registro hasta los programas que la procesan, muchas veces entre procesos o máquinas distintas; el preprocesamiento, que filtra la señal y la segmenta en ventanas, tramos de duración fija

que se analizan de a uno; la decodificación, en la que un modelo reconoce en cada ventana cuál de un conjunto de intenciones está ejecutando la persona; y la decisión, que convierte esa intención en una acción (Figura 1). Esa sucesión de etapas, en la que la salida de cada una es la entrada de la siguiente, es lo que en ingeniería de software se llama *pipeline*. El modelo de decodificación aprende antes de usarse, a partir de registros en los que se sabe qué imaginó la persona en cada repetición de la tarea: cada tramo de señal va acompañado de su respuesta correcta, llamada etiqueta (por ejemplo, «mano izquierda»), y el modelo ajusta sus parámetros hasta asociar los patrones de la señal con esas etiquetas; una vez entrenado, se aplica a señal nueva, cuya etiqueta desconoce. Un ejemplo sencillo es la imaginación motora (*motor imagery*), que consiste en imaginar un movimiento sin ejecutarlo: la persona imagina mover la mano izquierda o la derecha, ese acto produce patrones distinguibles en el EEG sobre la corteza motora, la región del cerebro que controla el movimiento voluntario, de cada hemisferio, y el decodificador clasifica cada ventana como izquierda o derecha para mover un cursor. Todo ese recorrido ocurre en tiempo real, muchas veces por segundo, y se ejecuta sobre una cadena de software que en la investigación es, en su mayor parte, de código abierto y compartida entre laboratorios. Esa cadena de software, y no el implante ni el modelo, es el objeto de esta investigación; la calidad de las decisiones que produce el decodificador es lo que el usuario percibe como funcionamiento o falla.

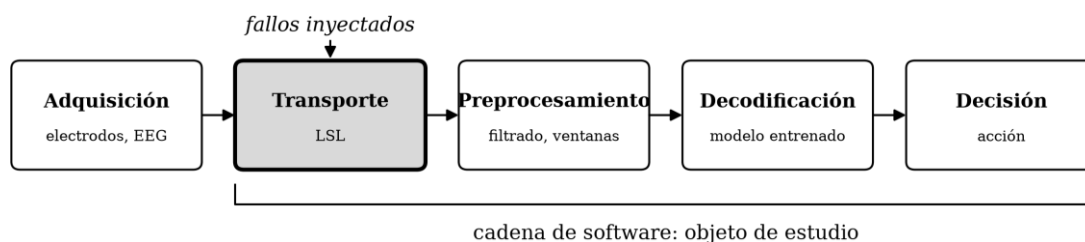


Figura 1. Etapas de una interfaz cerebro-computadora y objeto de estudio. Elaboración propia a partir de las etapas descritas por Nicolas-Alonso y Gomez-Gil (2012). En gris, la etapa sobre la que se inyectan los fallos; el corchete abarca la cadena de software que se estudia.

Para la persona que usa el sistema, el desempeño del decodificador no es un porcentaje sino una cantidad de comunicación por unidad de tiempo. Wolpaw et al. (2002) reportaron tasas máximas de transferencia de información de 10 a 25 bits por minuto para las BCI de su época y describieron que esa tasa depende de la exactitud de cada selección, del número de opciones disponibles y del tiempo que insume cada decisión; una caída de exactitud se paga, además, con las correcciones que el usuario debe hacer para deshacer las selecciones erróneas. Los sistemas recientes alcanzaron velocidades de comunicación muy superiores en tareas como la decodificación del habla, que se reportan en palabras por minuto y no son directamente comparables con aquellas tasas en bits, pero la relación de fondo se mantiene: una pérdida de exactitud en la decodificación se traduce, en una medida que depende de la tarea y del protocolo, en tiempo y esfuerzo adicionales para una persona con movilidad muy limitada. Por eso este trabajo mide el efecto de las fallas de software en el desempeño del decodificador y no solo en el estado técnico del sistema.

La decodificación de imaginaria motora tiene métodos y recursos consolidados. Blankertz et al. (2008) sistematizaron los patrones espaciales comunes (CSP, por su sigla en inglés), una técnica que combina los canales del EEG para resaltar las diferencias entre dos clases, habitualmente seguida de un análisis discriminante lineal (LDA), un clasificador que separa las clases con una frontera lineal; Lotte et al. (2018) revisaron diez

años de algoritmos de clasificación y concluyeron que esas combinaciones lineales seguían siendo referencias competitivas, y Roy et al. (2019) relevaron el uso creciente de *deep learning* (aprendizaje profundo) advirtiendo sobre la dificultad de comparar resultados entre estudios. Para hacerlos comparables existen recursos públicos: Tangermann et al. (2012) documentaron los datos de la competencia BCI Competition IV, entre ellos el conjunto 2a de imaginaria motora en nueve sujetos; Jayaram y Barachant (2018) desarrollaron MOABB, que estandariza la carga de esos conjuntos y la evaluación de algoritmos, y Gramfort et al. (2013) aportaron MNE-Python como biblioteca de análisis de EEG.

El software que ejecuta el sistema completo también se comparte. Schalk et al. (2004) presentaron BCI2000 y Renard et al. (2010) OpenViBE; más recientemente el ecosistema se desplazó hacia Python con BciPy (Memmott et al., 2021), MEDUSA (Santamaría-Vázquez et al., 2023), Timeflux (Clisson et al., 2019) y el marco con el ser humano en el lazo de Gemborn Nilsson et al. (2023), y Ali et al. (2024) publicaron BRAND para experimentos en lazo cerrado, en los que la salida del sistema realimenta al usuario en tiempo real, sobre datos intracorticales, es decir, registrados con electrodos implantados dentro de la corteza cerebral. Estos ocho artículos describen la arquitectura y las capacidades de cada herramienta, y varios reportan latencias en condiciones nominales; ninguno reporta el desempeño de un decodificador bajo una condición de falla del software. Estas herramientas son marcos de aplicación: sistemas completos que resuelven la presentación de estímulos, el procesamiento y la decodificación. Para recibir la señal del equipo de registro, la mayoría se apoya en una capa de transporte común, que se describe a continuación; BRAND es la excepción, porque utiliza para ese fin la base de datos en memoria Redis (Ali et al., 2024).

En la capa de transporte hay un estándar de facto: Lab Streaming Layer (LSL), una biblioteca de código abierto que permite que distintos programas, en una misma máquina o en varias conectadas en red, publiquen y reciban flujos de señal en tiempo real con sus marcas de tiempo sincronizadas, de modo que el equipo que registra el EEG, el que lo procesa y el que presenta los estímulos puedan trabajar como un solo sistema. El problema que resuelve es la sincronización: un experimento suele combinar varios equipos, como el EEG, un seguidor de la mirada o el programa que presenta las consignas, cada uno con su propio reloj, y analizar sus datos en conjunto exige saber con precisión de milisegundos qué ocurrió en cada uno al mismo tiempo; LSL lo resuelve en software, sin requerir un reloj de hardware común. Kothe et al. (2025) la describieron y reportaron, según su propio relevamiento, más de 2.300 menciones en artículos científicos, más de 150 clases de dispositivos compatibles y la integración en BCI2000, OpenViBE, MNE-Python, Timeflux y MEDUSA. La documentación del proyecto enumera integraciones con equipos de EEG de fabricantes como Brain Products, g.tec, ANT Neuro, BioSemi, Emotiv y OpenBCI, con seguidores de la mirada como Tobii y SR Research, con sistemas de captura de movimiento como Vicon y Qualisys, y con programas de presentación de estímulos como PsychoPy, E-Prime y Unity (Lab Streaming Layer, s. f.-b). Esa ubicuidad es la que hace relevante su comportamiento ante fallas: una propiedad no medida de LSL es una incertidumbre compartida por todos los experimentos que lo usan.

El artículo describe con detalle la arquitectura: los datos viajan por una red local convencional, con UDP, un protocolo sin confirmación de entrega, para descubrir los flujos disponibles, y una conexión TCP, que garantiza la entrega y el orden de los datos, para transmitirlos entre el productor, el programa que emite el flujo, y el consumidor, el que lo recibe; los relojes de las máquinas se sincronizan con un algoritmo derivado del

protocolo de tiempo de red (NTP), el mismo principio con el que las computadoras ajustan su hora, con una nueva estimación del desfase cada cinco segundos; los búferes (memorias intermedias que retienen datos mientras el consumidor no puede procesarlos) tienen capacidad configurable, de modo que si un consumidor deja de recibir momentáneamente los datos quedan retenidos hasta que pueda tomarlos; y si un productor desaparece, el consumidor lo busca periódicamente y se reconecta cuando reaparece, incluso en otra máquina con otra dirección. Los autores afirman que la biblioteca se somete periódicamente a pruebas de estrés con cientos de flujos y desconexiones, apagados y reconexiones aleatorios, y reportan como resultado de campo un *jitter*, es decir, una variación en el momento de llegada de los datos, con desviación estándar de aproximadamente 0,5 ms entre flujos de EEG y electromiografía, el registro de la actividad muscular. Lo que ese artículo no reporta es lo que este trabajo busca medir: qué proporción de muestras se pierde y cuánto tarda la recuperación cuando esos mecanismos actúan, y qué efecto tiene ese comportamiento sobre las etapas posteriores de la cadena, en particular sobre la decodificación. En ese sentido, las propiedades de tolerancia a fallos de LSL son, hasta donde alcanza esta revisión, autorreportadas y cualitativas.

Para dimensionar la actividad de esos proyectos se consultaron, el 16 de agosto de 2026, dos indicadores públicos de sus repositorios en GitHub, la plataforma donde se aloja y versiona su código: la cantidad de usuarios que los marcaron como favoritos (*estrellas*), medida habitual aunque indirecta de popularidad, y la fecha de su última modificación publicada. LSL registraba cambios recientes y una comunidad amplia, en consonancia con su rol de estándar de facto; BciPy y MEDUSA registraban cambios recientes pero comunidades reducidas, de entre veinte y algo más de ciento cincuenta estrellas; Timeflux no registraba cambios desde diciembre de 2024; y BRAND se

presentaba como infraestructura de un consorcio académico y no como estándar. Esos indicadores son indirectos y no miden la calidad del software ni la existencia de verificación, pero el panorama que muestran, una capa de transporte concentrada en una herramienta y una capa de aplicación fragmentada entre proyectos pequeños y de actividad desigual, refuerza el interés de contar con un protocolo de evaluación independiente y reutilizable, y sugiere que una debilidad no medida en la capa de transporte alcanzaría a la vez a todos los marcos que se apoyan en ella.

Una cadena en tiempo real impone restricciones que el análisis fuera de línea (*offline*) no tiene: en él, los datos ya grabados se procesan después y sin límite de tiempo, mientras que en una BCI en línea (*online*) cada decisión debe tomarse mientras la persona usa el sistema. Gemborn Nilsson et al. (2023) diseñaron su marco con cuatro módulos, un coordinador central, un módulo de cálculo construido sobre Timeflux que preprocesa, entrena y clasifica, y dos interfaces gráficas, comunicados por LSL para los datos y por *websockets*, un canal de comunicación bidireccional con el navegador, para la presentación de estímulos; al caracterizar el sistema documentaron que los propios dispositivos periféricos introducen retrasos considerables, entre 20 y 40 ms con 5 ms de *jitter* en un casco de EEG inalámbrico y entre 9 y 117 ms en los monitores, y señalaron como limitación de fondo que una decodificación más avanzada consume un tiempo que obliga a analizar los datos fuera de línea.

Wilson et al. (2010) habían definido la latencia de un sistema BCI, el tiempo que tarda un cambio en la entrada en producir un cambio causalmente relacionado en la salida, como la suma de las contribuciones de la conversión analógica-digital, el procesamiento y la salida, más el *jitter* entre bloques; midieron BCI2000 y una implementación en MATLAB sobre cuatro computadoras y dos sistemas operativos, encontraron latencias

de procesamiento de hasta 50 ms, salidas de video de 5 a 20 ms según el monitor y el sistema operativo, y un rango de 8 a 141 ms en MATLAB, y concluyeron que la elección del hardware y del sistema operativo tiene un efecto significativo sobre la temporalidad del experimento. Ambos trabajos miden sistemas que funcionan correctamente; ninguno introduce una falla para observar cómo la cadena responde.

De ahí se toman las tres propiedades del flujo de datos que este trabajo mide: la integridad, que todas las muestras emitidas lleguen; la temporalidad, que lleguen cuando corresponde y a intervalos regulares; y la disponibilidad, que el flujo no se interrumpa. La pérdida de muestras (*sample loss*) compromete la primera, el retraso y el *jitter* la segunda, y la desconexión la tercera. La literatura sobre robustez de la decodificación, en cambio, perturbó la señal y no el software: Urigüen y Garcia-Zapirain (2015) revisaron la eliminación de artefactos del EEG, es decir, de contaminaciones no cerebrales de la señal como parpadeos o movimientos musculares, y Meng et al. (2018) estudiaron cómo la reducción del número de electrodos afectó el desempeño de una BCI en línea; en ambos casos el transporte se tomó como dado.

Estas fallas no son situaciones excepcionales, sino parte del funcionamiento cotidiano de una BCI. Los cascos de EEG se conectan a la computadora muchas veces por enlaces inalámbricos, que pueden perder la señal por unos instantes; LSL transporta los datos por red, incluso entre computadoras distintas (Kothe et al., 2025), y una red puede congestionarse o cortarse y restablecerse; y el programa que procesa la señal comparte la máquina con otros procesos, de modo que puede demorarse si la computadora está ocupada. Cualquiera de estas situaciones altera el flujo de datos que recibe el pipeline. Ocurren, sin embargo, en capas distintas del sistema. La pérdida de alcance de un dispositivo inalámbrico afecta a la adquisición, antes de que la señal entre en LSL, y

puede eliminar muestras. La congestión de la red afecta al transporte, donde el protocolo TCP que LSL utiliza retransmite lo que se pierde, de modo que el efecto no es una muestra faltante sino más latencia y más variabilidad. Un consumidor demorado afecta al procesamiento: las muestras se acumulan en un búfer de capacidad finita que, según la documentación de LSL, descarta las más antiguas cuando se llena (Lab Streaming Layer, s. f.-a). Todas esas situaciones se manifiestan en el mismo lugar, la interfaz por la que el pipeline recibe el flujo, como muestras que faltan, que llegan tarde o a intervalos irregulares, o que dejan de llegar por un tiempo (Figura 2). Los cuatro modelos de fallo de esta investigación, pérdida de muestras, *jitter*, retraso y desconexión, se definen sobre esa interfaz: son perturbaciones observables del flujo que entra al pipeline, con orígenes posibles en más de una capa, y no se presentan como fallas propias del protocolo LSL.

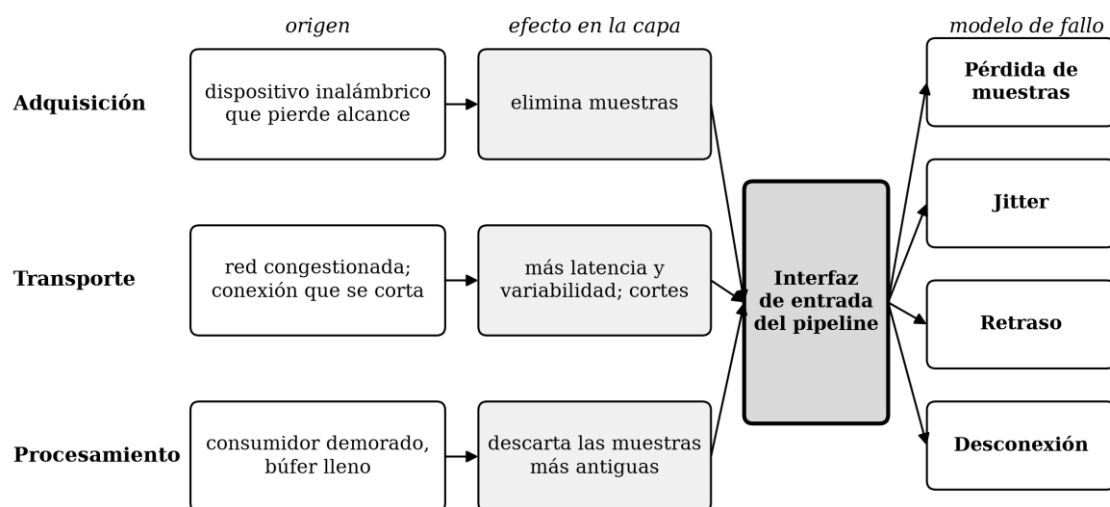


Figura 2. Orígenes de las perturbaciones en capas distintas y su manifestación común en la interfaz de entrada del pipeline. Elaboración propia. Los orígenes y efectos por capa son los descritos en el texto (Kothe et al., 2025; Lab Streaming Layer, s. f.-a); los cuatro modelos de fallo se definen sobre la interfaz de entrada, no sobre una capa en particular.

Fuera del dominio BCI, la ingeniería de software tiene un cuerpo de conocimiento maduro para este tipo de pregunta. Avizienis et al. (2004) fijaron la taxonomía de la

computación confiable (*dependable computing*), con la distinción entre falla, error y fallo de servicio, describieron la tolerancia a fallos como detección de errores más recuperación, e incluyeron los fallos en los que el sistema no señala la degradación de su servicio (*silent failures*). Natella et al. (2016) sistematizaron la inyección de fallos de software como el método que anticipa los escenarios de peor caso provocados por software defectuoso mediante la introducción deliberada de fallas, revisaron cómo evolucionaron sus técnicas en torno a tres criterios, la representatividad de las fallas inyectadas respecto de las reales, la eficiencia de las campañas y la usabilidad de las herramientas, y documentaron sus aplicaciones a la evaluación de sistemas tolerantes a fallos. Basiri et al. (2016) describieron cómo Netflix llevó esa idea a sistemas distribuidos en producción bajo el nombre de *chaos engineering* (ingeniería del caos), a partir de un servicio interno que apagaba al azar instancias de sus propios servidores para obligar a diseñar software que tolerara esas pérdidas, y la formalizaron en cuatro principios: formular una hipótesis sobre el comportamiento estable del sistema, variar eventos del mundo real, correr los experimentos en el entorno real y automatizarlos para que corran de forma continua. Los dos primeros principios son los que esta investigación adopta: una hipótesis sobre el estado estable, que aquí es el desempeño del pipeline sin fallos, y perturbaciones que reproducen eventos ordinarios de operación.

Los sistemas de procesamiento de flujos (*stream processing*), que como una BCI operan sobre secuencias continuas de eventos con restricciones temporales, fueron objeto de evaluaciones de este tipo. Karimov et al. (2018) compararon plataformas distribuidas con cargas de trabajo controladas en rendimiento y latencia, y Vogel et al. (2024) llevaron la comparación al terreno de la recuperación ante fallos: sobre Flink, Kafka Streams y Spark Structured Streaming desplegados en Kubernetes, inyectaron con la herramienta

Chaos Mesh la eliminación y la caída de instancias a intervalos regulares, y midieron el rendimiento, la latencia de los eventos en percentiles, el retraso acumulado del consumidor y el tiempo de recuperación, detectado automáticamente a partir de la media móvil y la desviación de las series. Encontraron que Flink fue el más estable y el de recuperación más rápida, que Kafka Streams mostró inestabilidad después de las fallas por su estrategia de rebalanceo, la forma en que redistribuye el trabajo entre las instancias que quedan, con casos en los que no se recuperó por completo antes de la falla siguiente, y que Spark se recuperó de manera consistente pero con mayor latencia.

Ese trabajo ofrece el molde metodológico que esta investigación adapta: familias de fallo, niveles de severidad, condición de referencia sin fallo, métricas de sistema medidas en el tiempo y un criterio explícito de recuperación. A diferencia de esos sistemas, una BCI tiene al final de la cadena un decodificador cuyas decisiones pueden compararse con lo que la persona realmente imaginó. Esa diferencia permite medir no solo si el sistema se recupera de una falla, sino si sigue decidiendo bien, y es la pieza que este trabajo agrega al molde metodológico.

Dicho de manera sencilla: ninguno de esos trabajos altera deliberadamente el transporte de la señal y observa, en el mismo experimento, dos cosas a la vez: si el sistema registra que algo anda mal y si el decodificador sigue acertando. De esos tres grupos de fuentes, plataformas de BCI, robustez de la decodificación e inyección de fallos en sistemas de propósito general, ninguno combina una falla inyectada en el transporte de señal con una medición simultánea del estado del sistema y del desempeño de un decodificador contra etiquetas conocidas. Para acotar esa afirmación se realizó en septiembre de 2026 una búsqueda en Google Scholar, Semantic Scholar, arXiv, PubMed e IEEE Xplore con términos que combinaron *fault injection*, *packet loss*, *jitter* y *Lab*

Streaming Layer con BCI y EEG. Los trabajos más próximos cubren una parte de esa combinación: Zheng et al. (2025) simuló cuatro modelos de pérdida de paquetes sobre registros intracorticales de macaco y midieron su efecto sobre un decodificador del movimiento del brazo, sin medir el estado del sistema, y Kothe et al. (2025) sometieron la propia capa de transporte a pruebas de estrés, sin medir su efecto sobre la decodificación. No se encontraron, en esas bases y con esos términos, trabajos que combinen ambas mediciones. Estudiarla no requiere hardware: MOABB provee la carga de los conjuntos públicos y el ecosistema de MNE-Python incluye componentes que reproducen una grabación como flujo LSL en tiempo real (Gramfort et al., 2013; Kothe et al., 2025), de modo que el mismo software que un laboratorio usaría con un amplificador y una persona puede ejecutarse con una fuente reproducible y variar solo aquello que se quiere estudiar.

El problema que aborda esta investigación es el desconocimiento sobre qué le ocurre a la decodificación de una BCI cuando el software que transporta la señal falla. No se sabe con qué magnitud la pérdida de muestras, el *jitter*, el retraso y la desconexión alteran la integridad, la temporalidad y la disponibilidad del flujo que recibe el decodificador, ni a partir de qué severidad esas alteraciones se traducen en una caída del desempeño de clasificación. Tampoco se sabe si hay condiciones en las que el proceso sigue activo, el flujo conectado y sin excepciones mientras la calidad de las decisiones cae (Figura 3), ni si la telemetría del propio sistema, las mediciones que registra sobre su propio estado, alcanza para detectar esa caída. Este último punto es el que más pesa cuando el usuario depende del sistema para comunicarse: una falla que el sistema informa puede gestionarse desde afuera; una que no informa solo se descubre por sus consecuencias.

		Estado de la infraestructura	
		Operativa	No operativa
Función del decodificador	No degradada	Concordancia funciona y decide bien	Divergencia: interrupción no operativa, decide bien
	Degradada	Divergencia: falla silenciosa operativa, decide mal	Concordancia falla visible y decide mal

Figura 3. Combinaciones posibles entre el estado de la infraestructura y la función del decodificador. Elaboración propia. Las dos celdas de divergencia son las que aborda el segundo interrogante; en gris, la falla silenciosa (*silent failure*), en la que el sistema no señala la degradación de su servicio (Avizienis et al., 2004).

La pregunta central es cómo se propagan fallos controlados de la capa de transporte de señal hacia las propiedades operativas y el desempeño funcional de un pipeline BCI en tiempo real, y en qué medida la telemetría del sistema permite detectar esa degradación. De ella se desprenden tres interrogantes, que se corresponden con las tres comparaciones del diseño experimental:

1. ¿Cómo afectan distintos tipos y severidades de fallo a la integridad de los datos, la disponibilidad, la latencia y el desempeño de decodificación del pipeline?
2. ¿Existen condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño del decodificador divergen?
3. ¿Puede un modelo de aprendizaje automático, entrenado para reconocer la degradación a partir de varias mediciones del sistema a la vez (la telemetría:

muestras recibidas, latencia, huecos, entre otras), detectarla mejor que un detector por umbrales, que dispara una alarma cuando una sola de esas mediciones supera un límite fijo?

Las tres admiten un resultado negativo: si la infraestructura absorbe los fallos, se documenta esa tolerancia; si no hay divergencia, ese resultado vale; y si un detector por umbrales rinde igual o mejor que el modelo, la complejidad adicional no se justifica. Asumir lo contrario de antemano habría condicionado el experimento hacia el resultado esperado.

La relevancia del problema proviene del lugar del software abierto en el campo: cuando una herramienta es gratuita, ubicua y la base sobre la que se validan y replican los resultados, una propiedad no medida de esa herramienta es una incertidumbre que arrastran todos los trabajos que corren sobre ella, y en una BCI del otro lado hay una persona que con frecuencia no dispone de otro canal para corregir una decisión errónea. Las BCI son una de las fronteras de la transformación digital en salud, y que su infraestructura sea confiable es condición para que lo construido sobre ella sea reproducible.

El planteo es conservador: somete al software a las condiciones que sus autores afirman tolerar, con datos públicos y sin hardware de adquisición, unidades de procesamiento gráfico ni acceso a pacientes. Toma de Vogel et al. (2024) la estructura del protocolo, de Natella et al. (2016) los criterios de diseño y de Basiri et al. (2016) la idea de verificar lo declarado; se aparta de ellos en el objeto, un pipeline BCI, y en la variable de respuesta, que incluye el desempeño de un decodificador. Sigue la tradición de la experimentación controlada en ingeniería de software (Wohlin et al., 2012), en la que el artefacto construido sirve para medir y no es el resultado. Lo que se espera obtener

es un protocolo reproducible con su banco de pruebas abierto, las curvas que relacionen cada condición de falla con su efecto sobre la infraestructura y la decodificación, y una comparación entre un mecanismo de alerta por umbrales y un modelo basado en telemetría.

Conviene dejar claro desde ahora qué no pretende el estudio. El enfoque Software-in-the-Loop reproduce el comportamiento del software real con señal grabada, pero no reproduce los electrodos, los amplificadores, la interferencia electromagnética ni la adaptación de la persona a un sistema que le devuelve retroalimentación. Las conclusiones se refieren, por lo tanto, al comportamiento del pipeline de software bajo las condiciones estudiadas y no a la resiliencia de una BCI clínica completa; extender el protocolo a hardware en el lazo queda como trabajo futuro.

El objetivo general de esta investigación es caracterizar empíricamente cómo se propagan fallos controlados de la capa de transporte de señal, en particular la pérdida de muestras, el *jitter*, el retraso y la desconexión, hacia las propiedades operativas y el desempeño de decodificación de un pipeline BCI en tiempo real basado en software de código abierto, y evaluar en qué medida la telemetría del sistema permite detectar esa degradación.

De ese objetivo general se derivan los siguientes objetivos específicos:

- Construir un banco de experimentación Software-in-the-Loop, es decir, el sistema de software real alimentado con señal grabada en lugar de una persona, que reproduzca señal electroencefalográfica pública como flujo en tiempo real sobre Lab Streaming Layer, con inyectores de fallos parametrizables, un decodificador de referencia y un recolector de telemetría.

- Cuantificar el efecto de cada tipo y severidad de fallo sobre la integridad, la temporalidad y la disponibilidad del flujo de datos, en comparación con la condición de referencia sin fallos.
- Medir el efecto de esos mismos fallos sobre el desempeño del decodificador de referencia y describir las curvas de degradación funcional resultantes.
- Identificar las condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño funcional del decodificador divergen.
- Comparar la capacidad de un detector basado en umbrales y la de un modelo de clasificación entrenado con telemetría multivariable para detectar la degradación funcional.
- Documentar el protocolo, los inyectores y los datos generados de manera que el estudio pueda replicarse y extenderse a otros pipelines.

Métodos

Diseño

La investigación adopta un enfoque cuantitativo, un alcance explicativo, sobre una base descriptiva, y un diseño experimental (Hernández Sampieri et al., 2014): mide variables numéricas y las analiza estadísticamente, busca establecer cómo cada fallo modifica el pipeline y, para ello, manipula deliberadamente la variable independiente, compara cada condición con una de control sin fallo y mide las mismas unidades bajo todas las condiciones (medidas repetidas). Por esa manipulación deliberada no corresponde la etiqueta de diseño no experimental transversal, habitual en los estudios por encuesta, que observan las variables sin intervenir sobre ellas.

Las variables independientes son el tipo de fallo (seis niveles en dos familias: cuatro distribuidos uniformemente en el tiempo y dos sincronizados con la ventana de decodificación) y su severidad (tres niveles por tipo); las dependientes siguen su efecto por el flujo de datos, la función del decodificador y la capacidad del sistema de advertirlo. Con la señal, el modelo y la máquina constantes, el efecto de cada fallo es la diferencia entre la métrica con y sin fallo en la misma corrida del mismo sujeto ($\Delta = Y \text{ con fallo} - Y \text{ de referencia}$), que cancela las diferencias entre sujetos.

Cada ejecución reproduce una corrida de la sesión de evaluación (48 ensayos, unos seis minutos y medio), sin fallo o con una condición fijada por una semilla que hace repetible su realización. Una primera etapa preespecificada (dos corridas por sujeto, referencia y doce condiciones uniformes, 234 ejecuciones) no mostró efecto funcional; a partir de ese resultado se incorporó la familia estructurada, con severidades fijadas antes de ejecutarla, y se extendió el diseño a cinco corridas, por lo que esa familia se interpreta como exploratoria. La campaña definitiva comprende 855 ejecuciones ($9 \text{ sujetos} \times 5$

corridas $\times$ 19 condiciones); la sexta corrida de tres sujetos se reservó para la prueba piloto, que fijó severidades, umbral y tamaño de ventana.

Participantes

La investigación no involucra participantes humanos. La muestra, no probabilística e intencional, es el pipeline de referencia descrito en Instrumentos, preferido a un marco completo como BciPy o MEDUSA porque sus capas de búfer y manejo de errores impedirían atribuir un efecto al transporte; las conclusiones se formulan para ese pipeline y esas condiciones.

La señal proviene del conjunto BCI Competition IV 2a (BNCI2014_001 en MOABB): EEG de 22 canales a 250 Hz de nueve sujetos, con dos sesiones de seis corridas de 48 ensayos de imaginación motora de cuatro clases (Tangemann et al., 2012). La primera sesión entrena el decodificador y la segunda se reproduce completa, pero solo sus 24 ensayos por corrida de mano izquierda o derecha se decodifican, porque CSP fue formulado para dos clases (Blankertz et al., 2008). El conjunto se distribuye como datos abiertos para la investigación (Tangemann et al., 2012) y sus sujetos se identifican solo por un número, por lo que el trabajo no requiere consentimiento informado adicional.

Instrumentos

El instrumento es un banco de experimentación Software-in-the-Loop de licencia abierta (Figura 4). El servicio de reproducción emite la grabación en bloques de diez muestras cada 40 ms, al ritmo de un equipo de registro y con su marca de tiempo nominal, mediante una pausa del sistema operativo seguida de una espera activa en los últimos 2 ms, y envía por un segundo flujo, sin perturbar, los marcadores de inicio de cada ensayo; el inyector actúa sobre el flujo de señal dentro del proceso emisor, de modo que lo único que varía entre ejecuciones es el fallo.

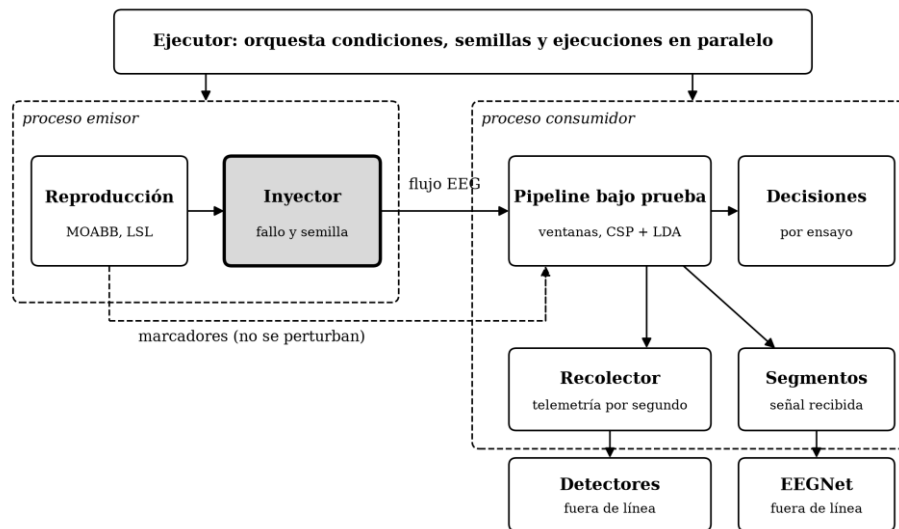


Figura 4. Componentes del banco de experimentación Software-in-the-Loop y flujo de datos entre ellos. Elaboración propia. En gris, el inyector de fallos; los recuadros discontinuos delimitan los dos procesos del sistema.

El pipeline bajo prueba corre en otro proceso. Para cada ensayo toma la ventana de 2 a 6 s desde el marcador, los cuatro segundos de la tarea, con un segundo adicional a cada lado para filtrar entre 8 y 30 Hz, y la clasifica con seis componentes CSP y LDA, por lo que la decisión se emite alrededor de un segundo después del fin del ensayo; si llegó menos de medio segundo de señal (125 muestras) o los datos no llegan en los tres segundos siguientes, la decisión es inválida (Figura 5). El banco guarda el segmento de señal de cada ensayo tal como llegó y la telemetría por segundo: muestras esperadas y recibidas, huecos, latencia, intervalo entre bloques, predicciones y excepciones.

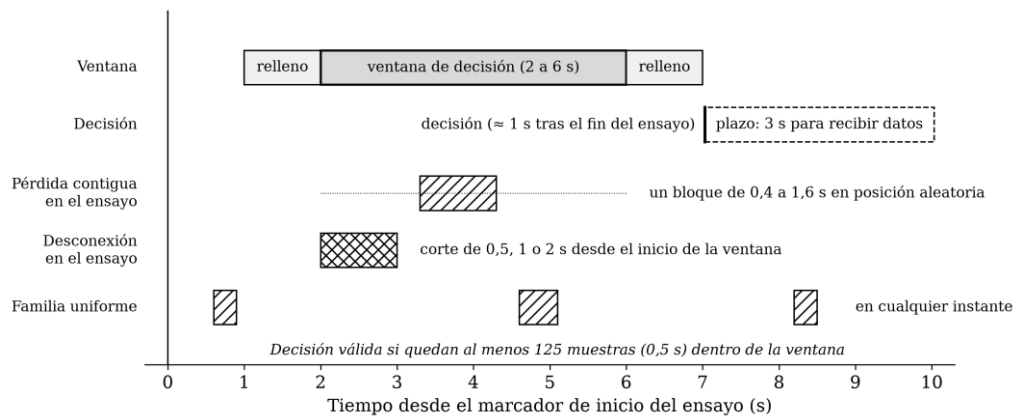


Figura 5. Línea de tiempo de un ensayo: ventana de decodificación, decisión y ubicación de los fallos de cada familia. Elaboración propia. Las posiciones de los fallos son ilustrativas.

Para verificar que el banco no altera la señal, el mismo decodificador se aplica también fuera de línea, directamente a la grabación y sin pasar por el banco; en la condición sin fallo, ambas exactitudes deben coincidir. Como análisis de robustez con una segunda familia de decodificador, EEGNet (Lawhern et al., 2018) se entrena por sujeto y se aplica después de cada ejecución, fuera de línea, a los segmentos guardados, con las muestras faltantes completadas con ceros; como esa entrada difiere de la de CSP y LDA en línea, se reaplicó CSP y LDA a los segmentos para comparar sus decisiones. El banco usa Python 3.12 (MNE-Python 1.13, MNE-LSL 1.14, MOABB 1.7, scikit-learn 1.9) y corrió en una máquina virtual dedicada (Amazon EC2 c6i.2xlarge, Ubuntu 24.04), porque el temporizador de Windows, de unos 15 ms, no permite inyectar un *jitter* de 10 ms; el efecto de las reproducciones en paralelo se controló contra la referencia ejecutada sola.

Los modelos de fallo se definen, siguiendo a Avizienis et al. (2004), como fallas en la interfaz de transporte que pueden o no propagarse a errores del estado y a fallos de servicio del decodificador (Tabla 1).

Tabla 1*Modelos de fallo de las dos familias: aplicación, severidades y tolerancia que miden*

Modelo	Dónde se aplica	Severidades	Tolerancia que mide
Pérdida de muestras	Se omiten muestras al azar	1, 5 y 10 %	Datos faltantes
<i>Jitter</i>	Demora aleatoria de cada bloque	Desviación de 10, 50 y 100 ms	Irregularidad temporal
Retraso	Demora constante de cada bloque	50, 100 y 250 ms	Latencia
Desconexión	El flujo se cierra y se recrea cinco veces por corrida, al azar	Cortes de 0,5, 1 y 3 s	Reconexión de LSL
Pérdida contigua en el ensayo	Bloque contiguo dentro de la ventana de cada ensayo	10, 25 y 40 % de la ventana	Hueco durante la decisión
Desconexión en el ensayo	Corte al inicio de la ventana de cada ensayo	Cortes de 0,5, 1 y 2 s	Reconexión durante la decisión

Nota. Elaboración propia. Las cuatro primeras filas son la familia uniforme y las dos últimas, la estructurada; las severidades toman como referencia el ensayo de cuatro segundos (mil muestras).

Con esos registros se calculan las variables dependientes; la Tabla 2 vincula cada una con su objetivo, su interrogante, su análisis y el lugar donde se informa. La métrica funcional principal es la *balanced accuracy*, el promedio de los aciertos de cada clase, porque los fallos invalidan ensayos de manera desigual entre clases: con 10 ensayos válidos de mano izquierda y 4 de derecha, responder siempre «izquierda» daría 71 % de exactitud simple y 50 % de exactitud balanceada, el nivel del azar.

Análisis de datos

Tabla 2

Correspondencia entre objetivos específicos, interrogantes, variables, análisis y resultados

Objetivo específico	Interrogante	Qué se mide	Cómo se analiza	Dónde se informa
1. Construir el banco	—	Error de reproducción; exactitud en el banco frente a la obtenida sobre la grabación	Descriptivo	Resultados, 1.er apartado
2. Cuantificar el efecto sobre el flujo	1	Integridad, temporalidad y disponibilidad	Friedman; Wilcoxon con Holm; r	Tabla 3 y Figura 6
3. Medir el efecto sobre el decodificador	1	<i>Balanced accuracy</i> , decisiones válidas y retardo de decisión	Friedman; Wilcoxon con Holm; r	Figura 7
4. Identificar la divergencia	2	Estado operativo frente a degradación funcional, segundo a segundo	Proporción de segundos divergentes	Resultados, 4.º apartado
5. Comparar los detectores	3	Precisión, sensibilidad, falsos positivos, precisión promedio y retardo	Validación dejando un sujeto fuera; prevalencia	Tabla 4
6. Documentar el estudio	—	Datos, guiones y semillas publicados	—	Resultados, último apartado

Nota. Elaboración propia. Los interrogantes son los numerados al final de la Introducción.

El análisis se realiza en Python con pandas, SciPy y scikit-learn. La unidad estadística es el sujeto: sus decisiones no son independientes entre sí, porque provienen del mismo cerebro, sesión y equipo, y tratarlas como independientes volvería significativa cualquier diferencia mínima (pseudorreplicación). Por eso las cinco corridas de cada sujeto y condición se resumen en su mediana, que resiste mejor que el promedio una corrida anómala, y esas nueve observaciones por condición ingresan a las pruebas; las tablas reportan su mediana.

Para el primer interrogante se usan pruebas no paramétricas apareadas, porque con nueve sujetos no puede verificarse la normalidad que suponen la t de Student o el análisis de varianza. Por tipo de fallo, una prueba de Friedman indica si alguna de las cuatro

condiciones (referencia y tres severidades) difiere, y una de Wilcoxon compara cada severidad con la referencia; sus valores p se ajustan con la corrección de Holm dentro de cada tipo, porque tres comparaciones elevan la probabilidad de un falso positivo por azar. Con nueve sujetos, el menor p corregido alcanzable es de aproximadamente 0,012, y el tamaño de efecto r (estadístico normal sobre la raíz del número de sujetos) alcanza su máximo, cercano a 0,96, cuando los nueve cambian en la misma dirección.

Para el segundo interrogante se cruzan, segundo a segundo, el estado operativo, la llegada de muestras sin excepciones del proceso, y la degradación funcional de CSP y LDA, una *balanced accuracy* sobre los últimos ocho ensayos inferior al mínimo observado en las corridas sin fallo del mismo sujeto; se prefirió ese umbral a un percentil, que marca por construcción una fracción de ventanas sanas como degradadas. Para el tercero, el detector por umbrales, que dispara cuando alguna variable de telemetría sale del rango de la referencia, se compara con regresión logística, *random forest* y *gradient boosting* con hiperparámetros fijos; todos se evalúan dejando un sujeto fuera en cada iteración, para evitar filtración de información, y su precisión se compara con la prevalencia de la degradación, la de un detector aleatorio. No se evalúa la anticipación, porque la ventana de ocho ensayos retrasa el inicio registrado de la degradación respecto del fallo.

Resultados

Los resultados se presentan en el orden de los objetivos específicos. Salvo indicación, las cifras son medianas entre sujetos de la mediana de cada sujeto sobre sus cinco corridas.

Construcción y verificación del banco de experimentación

El banco ejecutó las 855 ejecuciones de la campaña definitiva sin intervención manual ni ejecuciones fallidas, en 8,1 horas de reloj. En la referencia, el servicio de reproducción despertó para emitir cada bloque con un error cuyo percentil 99 fue de 1,3 microsegundos, y ejecutar doce reproducciones simultáneas modificó la latencia media en 0,04 ms y la desviación del intervalo entre bloques en 0,14 ms respecto de una ejecución aislada. La *balanced accuracy* de CSP y LDA en el banco sin fallo coincidió con la obtenida fuera de línea, aplicando el mismo decodificador directamente a la grabación, en los nueve sujetos, con valores entre 0,625 y 0,958, y la reaplicación de CSP y LDA a los segmentos guardados reprodujo el 99,0 % de las 20.511 decisiones válidas tomadas en línea. La primera etapa de la campaña (234 ejecuciones, familia uniforme) no mostró efecto funcional de ningún fallo.

Efecto de los fallos sobre la integridad, la temporalidad y la disponibilidad del flujo

Cada modelo de fallo modificó la variable sobre la que actúa, y las comparaciones de esa variable contra la referencia resultaron significativas en todas las severidades (Tabla 3); en la pérdida de muestras, el *jitter* y el retraso, con p corregida $\leq 0,012$ y $r = 0,96$. La pérdida de muestras no modificó la regularidad de llegada, y el *jitter* y el retraso no eliminaron muestras. Con desconexión, la relación de muestras recibidas (0,980, 0,980 y 0,954 para cortes de 0,5, 1 y 3 s) quedó por debajo de la fracción nominal no cortada (0,994, 0,987 y 0,961). El intervalo máximo entre llegadas fue de 1.561, 1.560 y 3.598

ms para cortes de 0,5, 1 y 3 s y de 1.565, 1.565 y 2.564 ms para desconexiones en el ensayo de 0,5, 1 y 2 s, igual para los cortes de 0,5 y 1 s en ambas familias (Figura 6). Con desconexión en el ensayo quedaron disponibles, en mediana, 620, 620 y 370 de las mil muestras de la ventana, y todas las decisiones fueron válidas.

Tabla 3

Variable perturbada por cada modelo de fallo, por severidad (mediana entre sujetos)

Tipo de fallo	Variable	Severidades	Ref.	Baja	Media	Alta
Pérdida de muestras	Muestras recibidas / esperadas	1, 5 y 10 %	1,000	0,990*	0,950*	0,900*
<i>Jitter</i>	Desv. del intervalo entre bloques (ms)	10, 50 y 100 ms	1,9	8,2*	30,8*	56,3*
Retraso	Latencia media (ms)	50, 100 y 250 ms	2,7	52,7*	102,7*	252,7*
Desconexión	Segundos sin datos por corrida	0,5, 1 y 3 s	0	3*	3*	13*
Pérdida contigua en el ensayo	Muestras recibidas / esperadas	10, 25 y 40 %	1,000	0,950*	0,876*	0,801*
Desconexión en el ensayo	Muestras recibidas / esperadas	0,5, 1 y 2 s	1,000	0,811*	0,811*	0,687*

Nota. Elaboración propia. Baja, media y alta: las tres severidades, en el orden indicado. El asterisco indica $p < 0,05$ en la comparación de Wilcoxon contra la referencia con corrección de Holm. La tabla completa de variables por condición se publica con el banco.

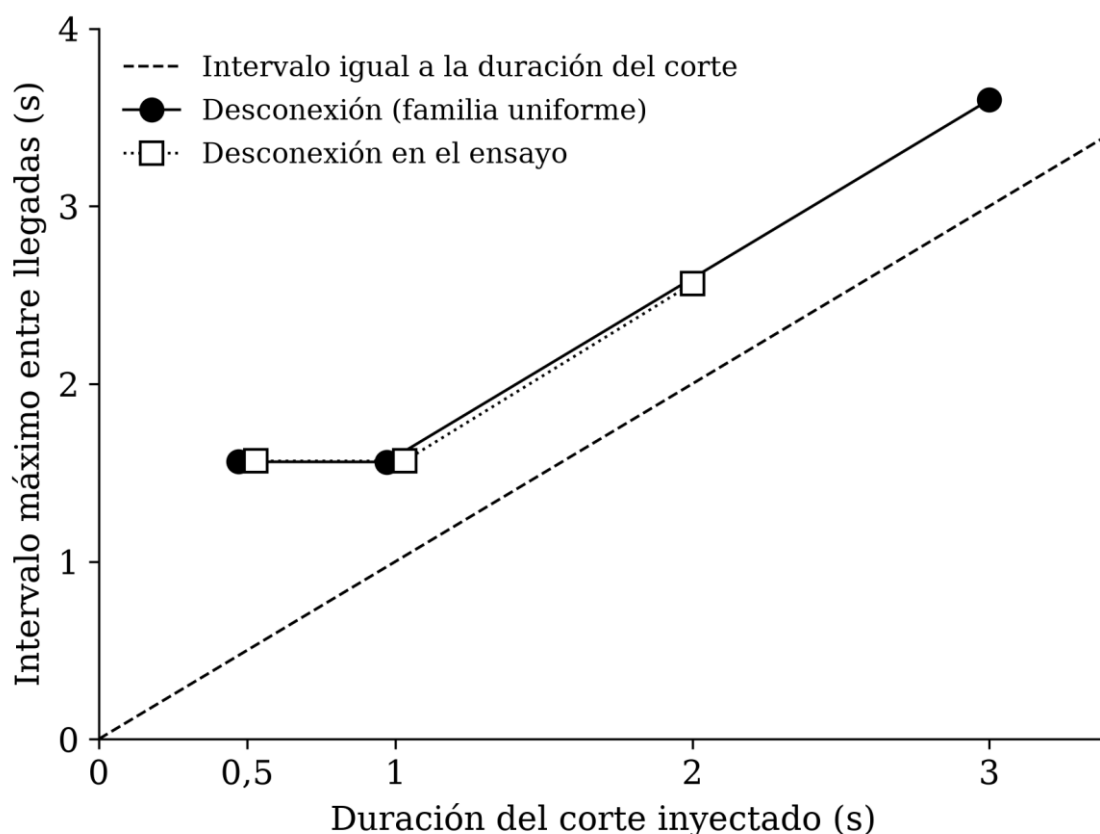


Figura 6. Intervalo máximo entre llegadas según la duración del corte inyectado, en las dos familias de desconexión. Elaboración propia. La línea discontinua marca un intervalo igual a la duración del corte. Los marcadores de 0,5 y 1 s se desplazaron levemente en horizontal para que no se superpongan.

Efecto de los fallos sobre el desempeño del decodificador

La *balanced accuracy* de referencia fue de 0,708 con CSP y LDA y de 0,750 con EEGNet, y ninguna de las 12 condiciones de la familia uniforme la modificó en ninguno de los dos decodificadores (Figura 7). En la familia estructurada, CSP y LDA obtuvo 0,708, 0,750 y 0,667 con pérdida contigua del 10, 25 y 40 % de la ventana y 0,750, 0,750 y 0,667 con desconexión en el ensayo de 0,5, 1 y 2 s, y EEGNet, 0,750, 0,750 y 0,708 y 0,583, 0,583 y 0,500. Resultaron significativas, para CSP y LDA, la pérdida contigua del 40 % de la ventana y la desconexión en el ensayo de 2 s, y para EEGNet, la pérdida contigua del 40 % de la ventana, la desconexión en el ensayo de 0,5 s, la desconexión en el ensayo de 1 s y la desconexión en el ensayo de 2 s. Las pruebas de Friedman de la

familia estructurada fueron significativas para ambos decodificadores (CSP y LDA: $\chi^2(3) = 14,09$, $p = 0,003$ y $\chi^2(3) = 16,75$, $p < 0,001$; EEGNet: $\chi^2(3) = 12,08$, $p = 0,007$ y $\chi^2(3) = 20,54$, $p < 0,001$). El retardo de decisión fue de 1.032 ms en la referencia, 1.082, 1.131 y 1.282 ms con retrasos de 50, 100 y 250 ms y 1.034, 1.047 y 1.077 ms con *jitter* de 10, 50 y 100 ms.

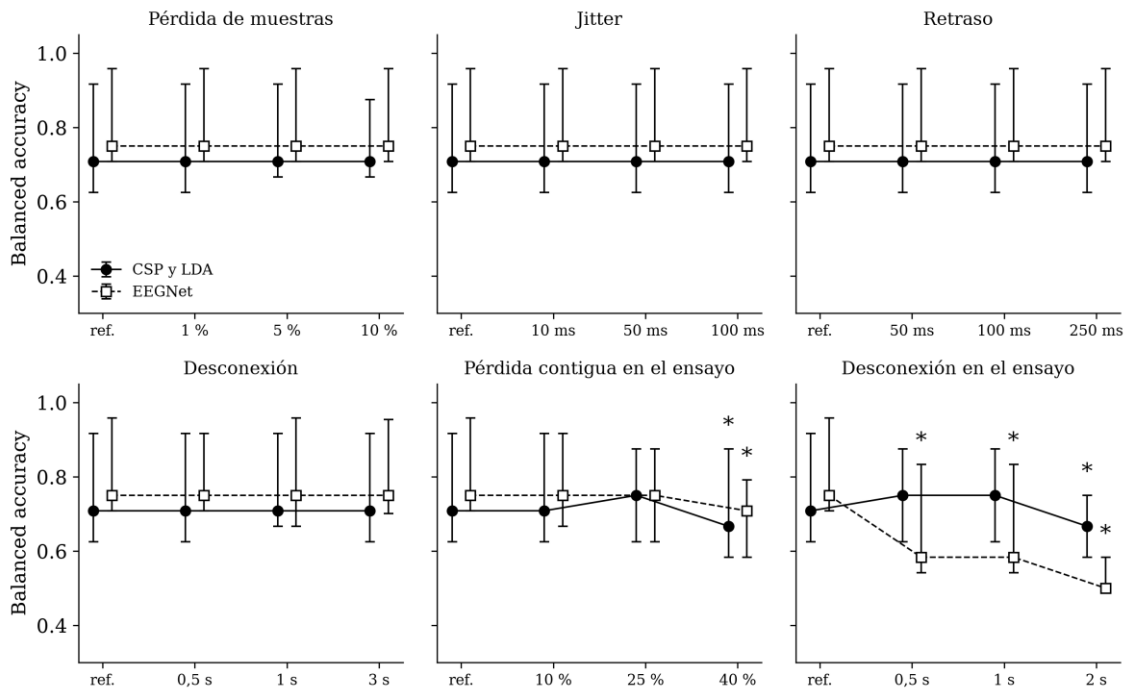


Figura 7. Balanced accuracy de los dos decodificadores por tipo y severidad de fallo. Elaboración propia. Marcadores: mediana entre sujetos; barras: rango intercuartílico; ref.: referencia. EEGNet se evaluó sobre los segmentos guardados por el pipeline. El asterisco indica $p < 0,05$ en la comparación de Wilcoxon contra la referencia con corrección de Holm.

Divergencia entre el estado operativo de la infraestructura y el desempeño funcional

La proporción de segundos en que la infraestructura se mostró operativa mientras CSP y LDA estaba degradado fue de 0,000 en las 18 condiciones. La divergencia inversa, segundos sin datos en los que el desempeño no cruzó el umbral, fue de 0,007, 0,007 y 0,035 con desconexiones de 0,5, 1 y 3 s, de 0,085, 0,085 y 0,206 con desconexiones en el ensayo de 0,5, 1 y 2 s y de 0,073 con pérdida contigua del 40 %, y nula en las demás condiciones. Se registraron 77 episodios de degradación funcional.

Detección de la degradación funcional a partir de la telemetría

De las 242.250 ventanas evaluadas, 3.255 correspondieron a degradación funcional (prevalencia 0,0134). La precisión de los cuatro detectores se ubicó entre 0,000 y 0,036 (Tabla 4), y la del detector por umbrales (0,010) fue inferior a la prevalencia. La regresión logística alcanzó la mayor sensibilidad (0,950) con una tasa de falsos positivos de 0,544, y *random forest*, la menor tasa de falsos positivos entre los detectores que emitieron alarmas (0,054) con una sensibilidad de 0,148.

Tabla 4

Desempeño de los detectores de degradación funcional con validación dejando un sujeto fuera

Detector	Precisión	Sensibilidad	F1	Tasa de falsos positivos	Precisión promedio	Retardo de detección (s)	Episodios detectados
Umbrales	0,010	0,425	0,019	0,587	0,012	4,0	77/77
Regresión logística	0,023	0,950	0,045	0,544	0,032	0,0	74/77
<i>Random forest</i>	0,036	0,148	0,058	0,054	0,024	5,0	66/77
<i>Gradient boosting</i>	0,000	0,000	0,000	0,000	0,029	—	0/77

Nota. Elaboración propia. Retardo de detección: mediana, en segundos, entre el primer segundo degradado de cada episodio y la primera alarma dentro de él; un guion indica que no se detectó ningún episodio. Precisión promedio: área bajo la curva de precisión y sensibilidad.

Documentación y replicabilidad

El banco, los guiones de análisis, las decisiones por ensayo, la telemetría por segundo y los segmentos de señal de todas las ejecuciones se publicaron con licencia MIT en <https://github.com/agustama27/bci-fault-bench> (versión 2.0), cada ejecución identificada por sujeto, corrida, condición y semilla, de modo que la realización de cada fallo es repetible.

Referencias

- Ali, Y. H., Bodkin, K., Rigotti-Thompson, M., Patel, K., Card, N. S., Bhaduri, B., Nason-Tomaszewski, S. R., Mifsud, D. M., Hou, X., Nicolas, C., Allcroft, S., Hochberg, L. R., Au Yong, N., Stavisky, S. D., Miller, L. E., Brandman, D. M., y Pandarinath, C. (2024). BRAND: A platform for closed-loop experiments with deep network models. *Journal of Neural Engineering*, 21(2), 026046.
<https://doi.org/10.1088/1741-2552/ad3b3a>
- Avizienis, A., Laprie, J.-C., Randell, B., y Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11-33.
<https://doi.org/10.1109/TDSC.2004.2>
- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., y Rosenthal, C. (2016). Chaos engineering. *IEEE Software*, 33(3), 35-41.
<https://doi.org/10.1109/MS.2016.60>
- Blankertz, B., Tomioka, R., Lemm, S., Kawanabe, M., y Müller, K.-R. (2008). Optimizing spatial filters for robust EEG single-trial analysis. *IEEE Signal Processing Magazine*, 25(1), 41-56. <https://doi.org/10.1109/MSP.2008.4408441>
- Card, N. S., Wairagkar, M., Iacobacci, C., Hou, X., Singer-Clark, T., Willett, F. R., Kunz, E. M., Fan, C., Vahdati Nia, M., Deo, D. R., Srinivasan, A., Choi, E. Y., Glasser, M. F., Hochberg, L. R., Henderson, J. M., Shahlaie, K., Stavisky, S. D., y Brandman, D. M. (2024). An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7), 609-618.
<https://doi.org/10.1056/NEJMoa2314132>

- Clisson, P., Bertrand-Lalo, R., Congedo, M., Victor-Thomas, G., y Chatel-Goldman, J. (2019). Timeflux: An open-source framework for the acquisition and near real-time processing of signal streams. En *Proceedings of the 8th Graz Brain-Computer Interface Conference 2019*. Verlag der Technischen Universität Graz.
- Gemborn Nilsson, M., Tufvesson, P., Heskebeck, F., y Johansson, M. (2023). An open-source human-in-the-loop BCI research framework: Method and design. *Frontiers in Human Neuroscience*, 17, 1129362.
<https://doi.org/10.3389/fnhum.2023.1129362>
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., y Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267.
<https://doi.org/10.3389/fnins.2013.00267>
- Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, M. del P. (2014). *Metodología de la investigación* (6.^a ed.). McGraw-Hill.
- Jayaram, V., y Barachant, A. (2018). MOABB: Trustworthy algorithm benchmarking for BCIs. *Journal of Neural Engineering*, 15(6), 066011.
<https://doi.org/10.1088/1741-2552/aadea0>
- Karimov, J., Rabl, T., Katsifodimos, A., Samarev, R., Heiskanen, H., y Markl, V. (2018). Benchmarking distributed stream data processing systems. En *2018 IEEE 34th International Conference on Data Engineering (ICDE)* (pp. 1507-1518). IEEE. <https://doi.org/10.1109/ICDE.2018.00169>
- Kothe, C., Shirazi, S. Y., Stenner, T., Medine, D., Boulay, C., Grivich, M. I., Artoni, F., Mullen, T., Delorme, A., y Makeig, S. (2025). The lab streaming layer for

synchronized multimodal recording. *Imaging Neuroscience*, 3.

<https://doi.org/10.1162/IMAG.a.136>

Lab Streaming Layer. (s. f.-a). *Frequently asked questions*. Lab Streaming Layer documentation. <https://labstreaminglayer.readthedocs.io/info/faqs.html>

Lab Streaming Layer. (s. f.-b). *Supported devices and tools*. Lab Streaming Layer documentation.

https://labstreaminglayer.readthedocs.io/info/supported_devices.html

Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., y Lance, B.

J. (2018). EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces. *Journal of Neural Engineering*, 15(5), 056013.

<https://doi.org/10.1088/1741-2552/aace8c>

Lotte, F., Bougrain, L., Cichocki, A., Clerc, M., Congedo, M., Rakotomamonjy, A., y

Yger, F. (2018). A review of classification algorithms for EEG-based brain-computer interfaces: A 10 year update. *Journal of Neural Engineering*, 15(3),

031005. <https://doi.org/10.1088/1741-2552/aab2f2>

Memmott, T., Koçanaoğulları, A., Lawhead, M., Klee, D., Dudy, S., Fried-Oken, M., y

Oken, B. (2021). BciPy: Brain-computer interface software in Python. *Brain-Computer Interfaces*, 8(4), 137-153.

<https://doi.org/10.1080/2326263X.2021.1878727>

Meng, J., Edelman, B. J., Olsoe, J., Jacobs, G., Zhang, S., Beyko, A., y He, B. (2018). A

study of the effects of electrode number and decoding algorithm on online EEG-based BCI behavioral performance. *Frontiers in Neuroscience*, 12, 227.

<https://doi.org/10.3389/fnins.2018.00227>

- Metzger, S. L., Littlejohn, K. T., Silva, A. B., Moses, D. A., Seaton, M. P., Wang, R., Dougherty, M. E., Liu, J. R., Wu, P., Berger, M. A., Zhuravleva, I., Tu-Chan, A., Ganguly, K., Anumanchipalli, G. K., y Chang, E. F. (2023). A high-performance neuroprosthesis for speech decoding and avatar control. *Nature*, 620(7976), 1037-1046. <https://doi.org/10.1038/s41586-023-06443-4>
- Natella, R., Cotroneo, D., y Madeira, H. S. (2016). Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3), 1-55. <https://doi.org/10.1145/2841425>
- Nicolas-Alonso, L. F., y Gomez-Gil, J. (2012). Brain computer interfaces, a review. *Sensors*, 12(2), 1211-1279. <https://doi.org/10.3390/s120201211>
- Renard, Y., Lotte, F., Gibert, G., Congedo, M., Maby, E., Delannoy, V., Bertrand, O., y Lécuyer, A. (2010). OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments. *Presence: Teleoperators and Virtual Environments*, 19(1), 35-53. <https://doi.org/10.1162/pres.19.1.35>
- Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., y Faubert, J. (2019). Deep learning-based electroencephalography analysis: A systematic review. *Journal of Neural Engineering*, 16(5), 051001. <https://doi.org/10.1088/1741-2552/ab260c>
- Santamaría-Vázquez, E., Martínez-Cagigal, V., Marcos-Martínez, D., Rodríguez-González, V., Pérez-Velasco, S., Moreno-Calderón, S., y Hornero, R. (2023). MEDUSA©: A novel Python-based software ecosystem to accelerate brain-computer interface and cognitive neuroscience research. *Computer Methods and*

Programs in Biomedicine, 230, 107357.

<https://doi.org/10.1016/j.cmpb.2023.107357>

Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., y Wolpaw, J. R. (2004).

BCI2000: A general-purpose brain-computer interface (BCI) system. *IEEE*

Transactions on Biomedical Engineering, 51(6), 1034-1043.

<https://doi.org/10.1109/TBME.2004.827072>

Tangemann, M., Müller, K.-R., Aertsen, A., Birbaumer, N., Braun, C., Brunner, C.,

Leeb, R., Mehring, C., Miller, K. J., Müller-Putz, G. R., Nolte, G., Pfurtscheller,

G., Preissl, H., Schalk, G., Schlögl, A., Vidaurre, C., Waldert, S., y Blankertz, B.

(2012). Review of the BCI Competition IV. *Frontiers in Neuroscience*, 6, 55.

<https://doi.org/10.3389/fnins.2012.00055>

Urigüen, J. A., y Garcia-Zapirain, B. (2015). EEG artifact removal—state-of-the-art and

guidelines. *Journal of Neural Engineering*, 12(3), 031001.

<https://doi.org/10.1088/1741-2560/12/3/031001>

Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., y Rabiser, R. (2024). A

comprehensive benchmarking analysis of fault recovery in stream processing

frameworks. En *Proceedings of the 18th ACM International Conference on*

Distributed and Event-Based Systems (DEBS '24) (pp. 171-182). ACM.

<https://doi.org/10.1145/3629104.3666040>

Willett, F. R., Kunz, E. M., Fan, C., Avansino, D. T., Wilson, G. H., Choi, E. Y.,

Kamdar, F., Glasser, M. F., Hochberg, L. R., Druckmann, S., Shenoy, K. V., y

Henderson, J. M. (2023). A high-performance speech neuroprosthesis. *Nature*,

620(7976), 1031-1036. <https://doi.org/10.1038/s41586-023-06377-x>

- Wilson, J. A., Mellinger, J., Schalk, G., y Williams, J. (2010). A procedure for measuring latencies in brain-computer interfaces. *IEEE Transactions on Biomedical Engineering*, 57(7), 1785-1797.
<https://doi.org/10.1109/TBME.2010.2047259>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., y Wesslén, A. (2012). *Experimentation in software engineering*. Springer. <https://doi.org/10.1007/978-3-642-29044-2>
- Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., y Vaughan, T. M. (2002). Brain-computer interfaces for communication and control. *Clinical Neurophysiology*, 113(6), 767-791. [https://doi.org/10.1016/S1388-2457\(02\)00057-3](https://doi.org/10.1016/S1388-2457(02)00057-3)
- Zheng, J., Li, Y., Chen, L., Wang, F., Gu, B., Sun, Q., Gao, X., y Zhou, F. (2025). Effects of packet loss on neural decoding effectiveness in wireless transmission. *Brain Sciences*, 15(3), 221. <https://doi.org/10.3390/brainsci15030221>